5. Multi-Plan Telecommunications Cost Optimization

Objective: To compare different mobile recharge plans based on data usage, call validity, SMS limits, and budget. Calculate extra charges and identify the most economical eligible plan.

1. Computational Thinking

- Decomposition: Divide the problem into plan details, user requirements, extra usage charges, and total cost.
- Pattern Recognition: Identify common features such as price, data limit, validity, and call charges.
- Abstraction: Consider only important plan details and user requirements.
- Algorithm Design: Compare all plans and select the lowest-cost plan that satisfies the requirements.

2. Algorithm

1. Start.
2. Enter the number of mobile plans.
3. Enter the user's required data, validity, and budget.
4. Store plan details in a list.
5. Read the price, data limit, validity, and extra data charge of each plan.
6. Calculate excess data usage.
7. Calculate the total cost of each plan.
8. Check whether the plan satisfies the data and validity requirements.
9. Check whether the total cost is within the budget.
10. Select the eligible plan with the lowest total cost.
11. Display all plan costs and the best plan.
12. If no plan is eligible, display "No suitable plan found".
13. Stop.

3. Variables and Data Types

Variable	Data type	Purpose
n	Integer	Number of plans
budget	Float	User's budget

req_data	Float	Required data
req_validity	Integer	Required validity
price	Float	Plan price
data	Float	Data limit
validity	Integer	Plan validity
extra_charge	Float	Extra usage charge
total_cost	Float	Final plan cost
plans	List	Stores plan details
eligible	Boolean	Checks plan eligibility
best_plan	Tuple	Stores the best plan

4. Operators

- Arithmetic: +, -, *
- Comparison: >=, <=, <
- Logical: and, or
- Assignment: =

5. Conditional Statements and Loops

- if: Checks whether a plan satisfies requirements.
- else: Displays when a plan is not eligible.
- for: Repeats the process for all mobile plans.
- if: Selects the cheapest eligible plan.

6. Pseudocode

START

INPUT number of plans

INPUT required data

INPUT required validity

INPUT budget

```
CREATE empty plan list
SET best_cost = infinity

FOR each plan:

    INPUT plan name
    INPUT price
    INPUT data limit
    INPUT validity
    INPUT extra data charge

    excess = MAX(0, required data - data limit)

    total_cost = price + (excess * extra data charge)

    IF data limit >= required data
      AND validity >= required validity
      AND total_cost <= budget:

        IF total_cost < best_cost:
            best_cost = total_cost
            best_plan = plan name

    ADD plan details to list

END FOR

IF best_cost is not infinity:
    DISPLAY best plan
```

DISPLAY best cost

ELSE:

DISPLAY "No suitable plan found"

STOP

7. Python Program

n = int(input("Enter number of plans: "))req_data = float(input("Enter required data (GB): "))req_validity = int(input("Enter required validity (days): "))budget = float(input("Enter your budget: "))plans = []best_cost = float('inf')best_plan = ""for i in range(n): print("\nEnter Plan", i + 1) name = input("Plan name: ") price = float(input("Plan price: ")) data = float(input("Data limit (GB): ")) validity = int(input("Validity (days): ")) charge = float(input("Extra data charge per GB: ")) excess = max(0, req_data - data) total = price + excess * charge plans.append((name, price, total)) if data >= req_data and validity >= req_validity: if total <= budget and total < best_cost: best_cost = total best_plan = nameprint("\n--- PLAN DETAILS ---")for p in plans: print(p[0], "Price:", p[1], "Total:", p[2])if best_plan != "": print("\nBest Plan:", best_plan)

8. Sample Output

Enter number of plans: 3

Required data: 2 GB

Required validity: 28 days

Budget: 300

Plan 1:

Price: 199

Data: 1.5 GB

Validity: 28

Extra charge: 50

Plan 2:

Price: 249

Data: 2.5 GB

Validity: 28

Extra charge: 0

Plan 3:

Price: 299

Data: 3 GB

Validity: 56

Extra charge: 0

--- PLAN DETAILS ---

Plan 1 Price: 199 Total: 224

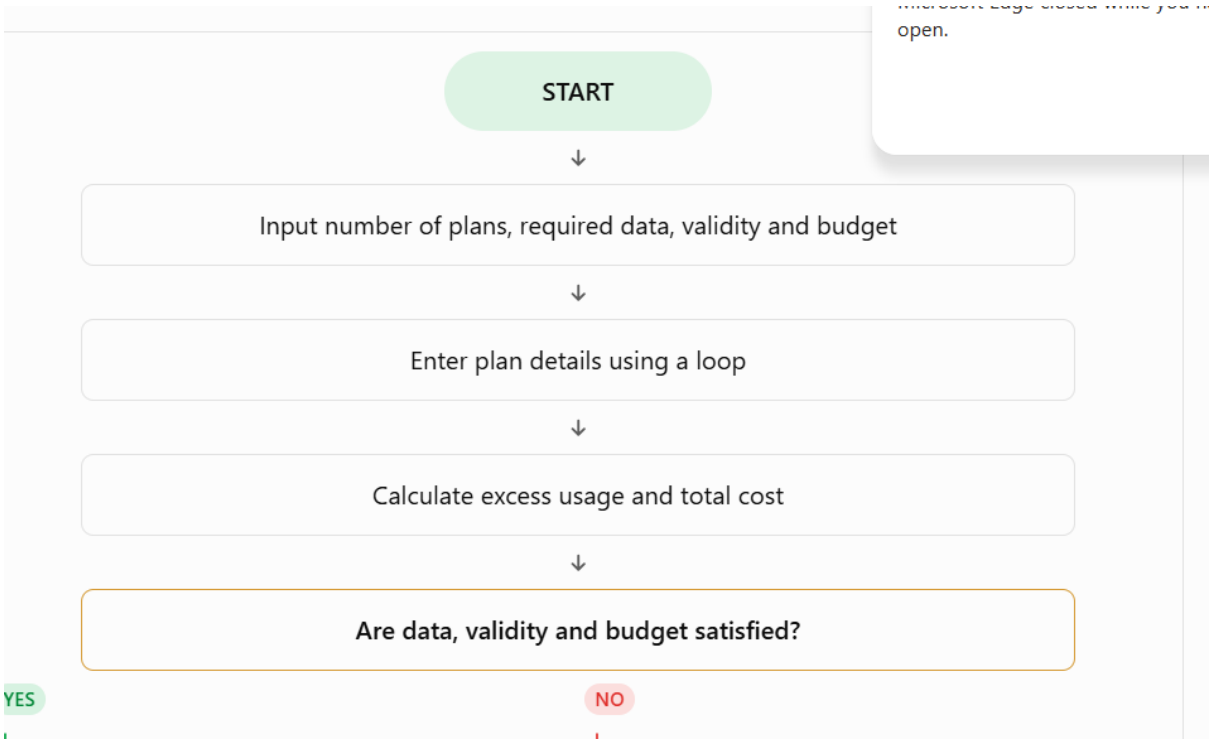
Plan 2 Price: 249 Total: 249

Plan 3 Price: 299 Total: 299

Best Plan: Plan 2

Total Cost: 249

9. Flowchart



START

Input number of plans, required data, validity and budget

Enter plan details using a loop

Calculate excess usage and total cost

Are data, validity and budget satisfied?

YES

Compare total cost and update best plan

NO

Skip plan

Repeat until all plans are checked

Display the cheapest eligible plan

STOP

START

Input number of plans, required data, validity and budget

Enter plan details using a loop

Calculate excess usage and total cost

Are data, validity and budget satisfied?

YES

Compare total cost and update best plan

NO

Skip plan

Repeat until all plans are checked

Display the cheapest eligible plan

STOP

10. Conclusion

This program compares multiple mobile recharge plans using data usage, validity, and budget constraints. It calculates excess charges and identifies the most economical eligible plan using Python lists, tuples, loops, and conditional statements.